More practice - Dijkstra and shortest paths

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	3
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Dijkstra and shortest paths | 4 questions | tap a title to jump to it

M	1. Compute shortest paths with blocked nodes	PracHub	2025-09-06
H	2. Minimize maximum height along a grid path	PracHub	2026-01-04
H	3. Find minimum threshold enabling grid path	PracHub	2026-02-11
H	4. Compute minimax grid path and network delay	PracHub	2026-01-22

STUDY PATH: question 122 of 167 (more practice)

Priority score 2.52 | Dijkstra and shortest paths | asked 409 days ago

MEDIUM | PracHub | Technical Screen | 1 of 4

★ TOP 75: rank 65 in the Medium and Hard priority list

★ Compute shortest paths with blocked nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 60/228

ABOUT THIS QUESTION

Topic: More practice - Dijkstra and shortest paths

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #743 Network Delay Time

LeetCode match: LC 743 Network Delay Time (Dijkstra with blocked nodes skipped)

Posted on PracHub: 2025-09-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/compute-shortest-paths-with-blocked-nodes

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.

Constraints

Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3],
    False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

APPROACH

Run Dijkstra (BFS if unweighted) from s, never relaxing into blocked nodes; blocked/unreachable nodes stay at infinity. O((V+E) log V).

HOW TO THINK ABOUT IT

Blocked nodes are simply not in the graph, so skip them when you queue neighbours and never start from one. After that it is plain BFS from the source: first discovery gives the shortest distance. State the edge cases: a blocked source makes everything unreachable, and directed edges must only be added in their given direction.

Basis: Unweighted shortest distances (BFS) with some nodes removed from the graph. Output: a distance per node, -1 for unreachable or blocked nodes; directed=True uses edges one way only.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, blocked, s):
    keep = []
    for u, v, w in edges:
        if u not in blocked and v not in blocked:
            keep.append((u, v, w))
    d = floyd_warshall(n, keep)  # all pairs
    return d[s]
```

TIME	SPACE
O(V^3)	O(V^2)

WHY IT IS SLOW

All-pairs distances cost O(V^3) and O(V^2) memory when one source is needed. A BFS from the source that never enters a blocked node is O(V + E).

OPTIMAL SOLUTION · BFS that never enters a blocked node

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges have weights.

BFS is no longer correct: use Dijkstra with a heap and skip blocked nodes the same way. O((V + E) log V). Directed and undirected differ only in whether you add the reverse edge.

FOLLOW-UP 2 Directed versus undirected.

For an undirected graph add each edge in both directions; for a directed one only as given. A node may be reachable in one case and not in the other, so state which you assume.

FOLLOW-UP 3 Nodes get blocked over time and you must keep distances up to date.

Decremental shortest paths is hard in general; for few updates rerun BFS, and for many offline queries process the changes in reverse order (unblocking only improves distances).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute shortest paths with blocked nodes
#
# Given a graph with nodes and edges and a designated
# source node s, compute the shortest distance from s to
# every other node. Some nodes are inaccessible (blocked)
# and cannot be traversed or reached; treat them as
# removed from the graph. Return a distance for each node
# where unreachable nodes (including blocked ones) are -1.
# Describe your algorithm, data structures, and time/space
# complexity, and note any differences for directed vs.
# undirected graphs. Overview: This question evaluates
# proficiency with graph algorithms, shortest-path and
# reachability concepts when nodes are removed or blocked,
# and the ability to analyze time and space complexity
# across directed and undirected graphs.Given n nodes
# labeled 0..n-1, edges, a source, and blocked nodes,
# return shortest unweighted distances from the source.
# Blocked and unreachable nodes must be -1. Set
# directed=True to treat edges as directed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges have weights.
# 2. Directed versus undirected.
# 3. Nodes get blocked over time and you must keep
# distances up to date.
#
# INTERVIEW NOTES
# Blocked nodes are simply not in the graph, so skip them
# when you queue neighbours and never start from one.
# After that it is plain BFS from the source: first
# discovery gives the shortest distance. State the edge
# cases: a blocked source makes everything unreachable,
# and directed edges must only be added in their given
# direction.
#
# BRUTE FORCE (say it first): Floyd-Warshall on the graph
# without the blocked nodes
# Compute all-pairs distances ignoring blocked nodes and
# read the row of the source. Correct and the oracle;
# cubic in the number of nodes. Time O(V^3). Space O(V^2).


import heapq
from collections import defaultdict, deque
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Floyd-Warshall on the graph
# without the blocked nodes | time O(V^3)
def blocked_distances_floyd(
    n, edges, source, blocked, directed
):
    INF = float("inf")
    bl = set(blocked)
    d = [
        [0 if i == j else INF for j in range(n)]
        for i in range(n)
    ]
    for u, v in edges:
        if u in bl or v in bl:
            continue
        d[u][v] = min(d[u][v], 1)
        if not directed:
            d[v][u] = min(d[v][u], 1)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    if source in bl:
        return [-1] * n
    return [
        -1 if x == INF or i in bl else x
        for i, x in enumerate(d[source])
    ]


# BASE QUESTION (optimal): BFS that never enters a blocked
# node
# Idea: Build adjacency lists (one direction only if
# directed). If the source is blocked return all -1.
def blocked_distances(n, edges, source, blocked,
                      directed=False):
    # BFS that treats blocked nodes as removed;
    # unreachable
    # and blocked nodes get -1
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        if not directed:
            adj[v].append(u)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in bad and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist


# FOLLOW-UP 1: The edges have weights.
# Answer: BFS is no longer correct: use Dijkstra with a
# heap and skip blocked nodes the same way. O((V + E) log
# V). Directed and undirected differ only in whether you
# add the reverse edge.
def blocked_distances_weighted(n, edges, source, blocked,
                               directed=False):
    # edges (u, v, w): BFS is wrong now, use Dijkstra
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        if not directed:
            adj[v].append((u, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
```

```
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if v not in bad and d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return [-1 if d == INF else d for d in dist]


if __name__ == "__main__":
    edges = [(0, 1), (1, 2), (0, 3), (3, 2), (2, 4)]
    free = blocked_distances(5, edges, 0, [])
    assert free == [0, 1, 2, 1, 3]
    # blocking 1 forces the route through 3
    assert blocked_distances(5, edges, 0, [1])[2] == 2
    # blocking 2 cuts node 4 off; blocked nodes are -1
    cut = blocked_distances(5, edges, 0, [2])
    assert cut == [0, 1, -1, 1, -1]
    start = blocked_distances(3, edges[:2], 0, [0])
    assert start == [-1, -1, -1]
    # directed edges only go one way
    path = [(0, 1), (1, 2)]
    one_way = blocked_distances(3, path, 2, [], True)
    assert one_way == [-1, -1, 0]
    weighted = [(0, 1, 5), (1, 2, 1), (0, 2, 10)]
    wd = blocked_distances_weighted
    assert wd(3, weighted, 0, []) == [0, 5, 6]
    assert wd(3, weighted, 0, [1])[2] == 10
```

STUDY PATH: question 123 of 167 (more practice)

Priority score 2.6 | Dijkstra and shortest paths | asked 289 days ago

HARD | IN APP LC 778, 1631 | PracHub | Technical Screen | 2 of 4

Minimize maximum height along a grid path

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 54/228

ABOUT THIS QUESTION

Topic: More practice - Dijkstra and shortest paths

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #778 Swim in Rising Water, LeetCode #1631 Path with Minimum Effort

LeetCode match: LC 778 Swim in Rising Water / LC 1631

Posted on PracHub: 2026-01-04 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/minimize-maximum-height-along-a-grid-path

QUESTION

Overview: This question evaluates algorithmic pathfinding and graph-modeling skills, specifically reasoning about grid-based route selection where the objective is defined by an aggregate maximum over visited cell heights.

APPROACH

Min-heap Dijkstra keyed by the max height seen so far (or binary search the cap + BFS, or union-find by sorted cell heights). O(mn log mn).

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, and a larger limit never makes things worse, so there are two equivalent plans: binary search the limit with a flood fill as the check, or run Dijkstra where the cost of a path is its maximum so far (extending a path never lowers it). Say the cost function out loud first: it is the maximum, not the sum.

Basis: LeetCode 778, Swim in Rising Water: the smallest maximum height such that a path from the top-left to the bottom-right cell exists using only cells at or below it. (The related LeetCode 1631 minimises the largest difference between adjacent cells, see the follow-up.)

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimize maximum height along a grid path
#
# This question evaluates algorithmic pathfinding and
# graph-modeling skills, specifically reasoning about
# grid-based route selection where the objective is
# defined by an aggregate maximum over visited cell
# heights.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
```

```
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over all simple paths from the top-
# left to the bottom-right, tracking the largest cell,
# with pruning when it cannot beat the best. Exact and the
# oracle; exponential in the grid size. Time exponential.
# Space O(path).


import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def min_max_path_brute(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]

    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
```

```
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


def reachable(H, limit):
    # flood fill using only cells with height <= limit
    m, n = len(H), len(H[0])
    if H[0][0] > limit:
        return False
    seen, stack = {(0, 0)}, [(0, 0)]
    while stack:
        r, c = stack.pop()
        if (r, c) == (m - 1, n - 1):
            return True
```

```
        for a, b in around(r, c, m, n):
            if (a, b) not in seen and H[a][b] <= limit:
                seen.add((a, b))
                stack.append((a, b))
    return False


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H, with_path=False):
    # Dijkstra where a path costs the MAX of its cells
    m, n = len(H), len(H[0])
    best, parent = {(0, 0): H[0][0]}, {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))


# ALTERNATIVE: binary search the answer
def min_max_path_bsearch(H):
    # binary search the answer over the sorted heights
    heights = sorted({h for row in H for h in row})
    lo, hi = 0, len(heights) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if reachable(H, heights[mid]):
            hi = mid
        else:
            lo = mid + 1
```

```
    return heights[lo]


# FOLLOW-UP 1: The cost is the largest difference between
# adjacent cells on the path (LeetCode 1631).
# Answer: Same Dijkstra with max, but the step cost is
# abs(height difference) instead of the cell value. The
# binary-search version also works: the check allows an
# edge only if the difference is at most the limit.
def min_effort_path(H):
    # LeetCode 1631: cost = the largest height DIFFERENCE
    m, n = len(H), len(H[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in around(r, c, m, n):
            nxt = max(cost, abs(H[a][b] - H[r][c]))
```

```
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))


if __name__ == "__main__":
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert min_max_path(H) == min_max_path_bsearch(H)
    cost, path = min_max_path(H, True)
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    assert max(H[r][c] for r, c in path) == cost
    assert min_max_path([[7]]) == 7
    grid = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    assert min_max_path(grid) == 6
    assert min_max_path_bsearch(grid) == 6
    assert min_effort_path([[1, 2, 2], [3, 8, 2],
                            [5, 3, 5]]) == 2
```

FUNCTION AND TEST CASES

Sample call

```
solution([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Test cases

Input	Expected
`([[1, 2, 3], [4, 5, 6], [7, 8, 9]],)`	`9`
`([[8, 4, 7], [6, 5, 9], [3, 2, 1]],)`	`8`

ALSO REPORTED · 1 more

Path Minimizing the Maximum Cell (Swim in Rising Water) | 1Point3Acres | onsite-coding | asked 2026-07-08 | Low

https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter. Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

Preparation: Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation. Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY PATH: question 124 of 167 (more practice)

Priority score 2.1 | Dijkstra and shortest paths | asked 251 days ago

HARD | IN APP LC 1631, 778 | PracHub | Technical Screen | 3 of 4

Find minimum threshold enabling grid path

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 53/228

ABOUT THIS QUESTION

Topic: More practice - Dijkstra and shortest paths

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1631 Path with Minimum Effort, LeetCode #778 Swim in Rising Water

LeetCode match: LC 1631 Path With Minimum Effort / LC 778 Swim in Rising Water (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-06-01)

Original: https://prachub.com/coding-questions/find-minimum-threshold-enabling-grid-path

QUESTION

Overview: This question evaluates a candidate's understanding of grid pathfinding, reachability under constraints, and algorithmic optimization, focusing on skills such as graph traversal and handling threshold-based feasibility.

APPROACH

Binary search the threshold and BFS/DFS for reachability, or run Dijkstra/union-find where path cost is the max cell value along the path.

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, and a larger limit never makes things worse, so there are two equivalent plans: binary search the limit with a flood fill as the check, or run Dijkstra where the cost of a path is its maximum so far (extending a path never lowers it). Say the cost function out loud first: it is the maximum, not the sum.

Basis: LeetCode 778, Swim in Rising Water: the smallest threshold such that a path from the top-left to the bottom-right cell exists using only cells at or below it. (The related LeetCode 1631 minimises the largest difference between adjacent cells, see the follow-up.)

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find minimum threshold enabling grid path
#
# This question evaluates a candidate's understanding of
# grid pathfinding, reachability under constraints, and
# algorithmic optimization, focusing on skills such as
# graph traversal and handling threshold-based
# feasibility.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
```

```
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over all simple paths from the top-
# left to the bottom-right, tracking the largest cell,
# with pruning when it cannot beat the best. Exact and the
# oracle; exponential in the grid size. Time exponential.
# Space O(path).


import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def min_max_path_brute(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]

    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# ALSO: the Dijkstra version
def best_path(H, step_cost, start_cost):
    # Dijkstra where a path costs the MAX of its steps;
    # the
    # first time a cell leaves the heap its cost is final
    m, n = len(H), len(H[0])
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, step_cost(H, r, c, a, b))
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))
    return None, []


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H):
    # cost of a path = its largest cell value
    # (water rising over time is the same problem)
    def step(H, r, c, a, b):
        return H[a][b]
    return best_path(H, step, H[0][0])


# FOLLOW-UP 1: The cost is the largest difference between
# adjacent cells on the path (LeetCode 1631).
# Answer: Same Dijkstra with max, but the step cost is
# abs(height difference) instead of the cell value. The
# binary-search version also works: the check allows an
# edge only if the difference is at most the limit.
def min_effort_path(H):
    # LeetCode 1631: cost = the largest height difference
    # between two neighbouring cells on the path
    def step(H, r, c, a, b):
        return abs(H[a][b] - H[r][c])
    return best_path(H, step, 0)


if __name__ == "__main__":
    H = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    cost, path = min_max_path(H)
    # 0 -> 1 -> 3 -> 7 -> 6 or via 2, 5, 6: the max is 6
    assert cost == 6
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    assert max(H[r][c] for r, c in path) == cost
    assert min_max_path([[5]]) == (5, [(0, 0)])
    effort, path2 = min_effort_path([[1, 2, 2], [3, 8, 2],
                                     [5, 3, 5]])
    assert effort == 2
    assert min_effort_path([[7]]) == (0, [(0, 0)])
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 2], [1, 3]])
```

Test cases

Input	Expected
`([[0, 2], [1, 3]],)`	`3`
`([[0, 100, 2], [1, 2, 2], [3, 4, 1]],)`	`2`

STUDY PATH: question 125 of 167 (more practice)

Priority score 2.02 | Dijkstra and shortest paths | asked 271 days ago

HARD | PracHub | Technical Screen | 4 of 4

Compute minimax grid path and network delay

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 99/228

ABOUT THIS QUESTION

Topic: More practice - Dijkstra and shortest paths

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #1631 Path with Minimum Effort, LeetCode #743 Network Delay Time

LeetCode match: LC 1631 Path With Minimum Effort + LC 743 Network Delay Time (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/compute-minimax-grid-path-and-network-delay

QUESTION

Overview: This question evaluates algorithmic problem-solving in shortest-path and path-optimization contexts, specifically minimax pathfinding on a grid and single-source reachability/timing in a weighted directed graph, testing graph modeling, alternative path cost metrics, edge-case handling, and working with large numeric weights.

APPROACH

Part 1: Dijkstra (or binary search + BFS) where cost is the max edge/cell along the path. Part 2: Dijkstra from the source; answer is the max distance, or -1 if some node is unreachable.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Part 1: ask what the path cost is (the maximum cell value along the path, including the start), the allowed moves (4-directional) and the endpoints (top-left to bottom-right). Part 2: ask whether the graph is directed, what to return if some node is unreachable (-1), and the size of weights. The scraped solution covers only part 1.

BRUTE FORCE TO SAY OUT LOUD

Part 1: DFS over all paths, O(4^(mn)). Part 2: Bellman-Ford, which is fine for small graphs.

OPTIMAL IDEA

Part 1: Dijkstra where the cost of a path is the maximum seen so far, with a min-heap keyed on that maximum: new_cost = max(cost, cell). O(mn log mn). Equivalent: binary search the cap and BFS, or sort cells and union-find until start and end connect. Part 2: Dijkstra from the source; the time for a signal to reach everyone is the largest shortest distance, or -1 if any node is unreachable. O((V + E) log V).

DRY-RUN THESE CASES

Part 1: [[1,10,6],[1,3,2],[7,2,2]] gives 3 (as in the example), a 1 x 1 grid, a grid where the best path takes a detour. Part 2: source with no outgoing edges, a disconnected node, parallel edges.

LIKELY FOLLOW-UPS

Part 1 with the cost as the sum minus something, or paths that may revisit cells. Part 2 with large weights (use 64-bit integers).

PITFALLS

Using the sum instead of the maximum when relaxing. Not counting the start cell's own value.

HOW TO THINK ABOUT IT

Both are 'shortest path' with a different notion of cost. For the grid the cost of a path is its maximum, which still works in Dijkstra: the best way to reach a cell is the one with the smallest maximum, and extending a path can never lower it. For the network, Dijkstra from the source gives every arrival time, and the answer is the largest of them (-1 if some node is never reached).

Basis: Two parts: LeetCode 1631-style minimax path (path cost = the largest cell value on the path, start cell included) and LeetCode 743, Network Delay Time (time for a signal to reach every node, or -1).

BRUTE FORCE · Part 1: DFS over every simple path

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Dijkstra where the cost of a path is its maximum cell (or a union-find over cells added in increasing order) is O(m * n log(m * n)).

BRUTE FORCE · Part 2: Bellman-Ford from the source

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, source):
    dist = [INF] * n
    dist[source] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    worst = max(dist)
    return -1 if worst == INF else worst
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

V - 1 rounds over all edges, O(V * E). Dijkstra with a heap is O((V + E) log V) for non-negative weights.

OPTIMAL SOLUTION · Part 1: Dijkstra where the cost of a path is its maximum cell

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Part 2: Dijkstra from the source, answer is the largest distance

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 1 with a union-find (sort cells, add them one by one).

Add cells in increasing order of height, union neighbours already added, and stop when the start and the goal are connected: that height is the answer. O(mn log mn) for the sort.

FOLLOW-UP 2 Part 2: weights are huge.

Use 64-bit integers (Python is fine); keep sums in a wide type, and with weights up to 10^9 and 10^5 edges a path can reach 10^14. **Cost:** unchanged O((V + E) log V); sums need 64-bit (or Python) integers.

FOLLOW-UP 3 Part 2: the signal can be blocked at some nodes, or each node adds a processing delay.

A node delay is added to the edge cost leaving the node (or modelled by splitting each node into in and out with an edge of the delay). The same Dijkstra then applies.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute minimax grid path and network
# delay
#
# This question evaluates algorithmic problem-solving in
# shortest-path and path-optimization contexts,
# specifically minimax pathfinding on a grid and single-
# source reachability/timing in a weighted directed graph,
# testing graph modeling, alternative path cost metrics,
# edge-case handling, and working with large numeric
# weights.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 1 with a union-find (sort cells, add them one by
# one).
# 2. Part 2: weights are huge.
# 3. Part 2: the signal can be blocked at some nodes, or
# each node adds a processing delay.
#
# INTERVIEW NOTES
# Both are 'shortest path' with a different notion of
# cost. For the grid the cost of a path is its maximum,
# which still works in Dijkstra: the best way to reach a
# cell is the one with the smallest maximum, and extending
# a path can never lower it. For the network, Dijkstra
# from the source gives every arrival time, and the answer
# is the largest of them (-1 if some node is never
# reached).
#
# BRUTE FORCE (say it first): Part 1: DFS over every
# simple path
# Try all simple paths from the top-left to the bottom-
# right and take the smallest maximum cell value. Exact
# and the oracle; exponential in the grid size. Time
# exponential. Space O(path).


import heapq


# BRUTE FORCE (say it first): Part 1: DFS over every
# simple path | time exponential
def minimax_path_dfs(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]
```

```
    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
```

```
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def neighbors(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# BASE QUESTION (optimal): Dijkstra where the cost of a
# path is its maximum cell
# Idea: Heap of (best maximum so far, cell). Pop the cell
# with the smallest bottleneck, and for each neighbour the
# new bottleneck is max(current, neighbour). Improve if
# smaller.
def minimax_path(H):
    # Part 1: the path whose highest cell is as low as
    # possible
    m, n = len(H), len(H[0])
```

```
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in neighbors(r, c, m, n):
            # the cost of a path is its maximum, never a
            # sum
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))
    return -1


# PART 2: network delay
def network_delay(n, edges, source, node_delay=None):
    # Part 2: when does the signal reach every node?
    # edges are (u, v, weight); -1 if some node is
    # unreachable
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        # a processing delay is paid when leaving a node
        pause = node_delay[u] if node_delay else 0
        for v, w in graph[u]:
            if d + pause + w < dist[v]:
                dist[v] = d + pause + w
                heapq.heappush(heap, (dist[v], v))
```

```
    worst = max(dist)
    return -1 if worst == INF else worst


# FOLLOW-UP 1: Part 1 with a union-find (sort cells, add
# them one by one).
```

```
# Answer: Add cells in increasing order of height, union
# neighbours already added, and stop when the start and
# the goal are connected: that height is the answer. O(mn
# log mn) for the sort.
def minimax_path_union_find(H):
    # the same answer: add cells from lowest to highest
    # and
    # stop when the start and the goal are connected
    m, n = len(H), len(H[0])
    parent = {}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    cells = sorted((H[r][c], r, c)
                   for r in range(m) for c in range(n))
    for h, r, c in cells:
        parent[(r, c)] = (r, c)
        for a, b in neighbors(r, c, m, n):
            if (a, b) in parent:
                parent[find((r, c))] = find((a, b))
        if (0, 0) in parent and (m - 1, n - 1) in parent:
            if find((0, 0)) == find((m - 1, n - 1)):
                return h
    return -1


if __name__ == "__main__":
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert minimax_path(H) == 2
    assert minimax_path(H) == minimax_path_union_find(H)
    assert minimax_path([[5]]) == 5
    edges = [(0, 1, 2), (1, 2, 3), (0, 2, 10)]
    assert network_delay(3, edges, 0) == 5
    # a node that cannot be reached
    assert network_delay(4, edges, 0) == -1
    # huge weights are fine with Python integers
    big = [(0, 1, 10 ** 9), (1, 2, 10 ** 9)]
    assert network_delay(3, big, 0) == 2 * 10 ** 9
    # each node adds a processing delay of 1
    slow = network_delay(3, big, 0, [1, 1, 1])
    assert slow == 2 * 10 ** 9 + 2
```

FUNCTION AND TEST CASES

Signature

```
def solution(H: list[list[int]]) -> int:
SAMPLE INPUT
[[1, 10, 6], [1, 3, 2], [7, 2, 2]]
```

Test cases

Input	Expected
`[[1, 10, 6], [1, 3, 2], [7, 2, 2]]`	`3`
`[[5, 1, 7], [4, 8, 2], [3, 6, 0]]`	`6`